PROJECT DOCUMENTATION

SCOMS

School & College Management System

A multi-tenant, multi-portal Education ERP platform covering institution administration, staff operations, and the student & parent experience — built on a metadata-driven architecture so new institutions, fee structures, reports and certificates never require code changes.

Super Admin | Institute Admin | Staff Portal | Student Portal

Generated 31 August 2026

Node.js · Express · PostgreSQL · Prisma ORM · React 19 · Vite · Tailwind CSS

Table of Contents

1. **System Overview**
2. **Technology Stack**
3. **System Architecture Diagram**
4. **Database Design Philosophy**
5. **Module 1 — Super Admin Portal**
6. **Module 2 — Institute Admin Portal**
    - 6.1 Core Academic & Operational Features
    - 6.2 Dynamic Report Engine
    - 6.3 Dynamic Certificate Engine
    - 6.4 Dynamic Fee Engine
7. **Module 3 — Staff Portal**
8. **Module 4 — Student & Parent Portal**
9. **Security & Multi-Tenancy Model**
10. **Glossary**

1. System Overview

SCOMS (School & College Management System) is a multi-tenant Education ERP platform serving four distinct types of users through four independently-deployed portals, each backed by its own Express.js server, sharing one PostgreSQL database and one common code library ("shared").

What "Multi-Tenant" Means Here

Every institution that signs up (a school, college, or university) is one tenant. All of that institution's data — students, staff, fee structures, exams, certificates — is scoped strictly to that institution via an institutionId derived from the logged-in user's JWT token, never from anything the browser sends. One institution can never see or affect another institution's data, even though they all run on the exact same servers and the exact same database.

The Four Portals at a Glance

Portal	Audience	Backend Port	Primary Purpose
Super Admin	SCOMS platform operators	5001	Onboard institutions, manage the platform, configure system-wide catalogs (reports,
Institute Admin	A single institution's administrators	5002	Run the institution day-to-day — academics, staff, students, fees, exams, communication
Staff	Teachers & other institution staff	5003	Attendance, assignments, exams, grading, communication with students/parents
Student	Students & their parents	5004	View academics, attendance, results, pay fees, download certificates, communicate

Design Principle: Metadata-Driven, Not Hard-Coded

A recurring theme across the newest parts of this system (the Report Engine, Certificate Engine, and Fee Engine, documented in Module 2) is that institution-specific configuration lives in database rows, not in code. An institution can define its own fee heads, its own report catalog visibility, and its own certificate wording — all through the UI — without a developer ever touching the codebase or altering a database table.

2. Technology Stack

Backend (×4 — one per portal)

- **Runtime:** Node.js with Express.js — four separate apps (SuperAdmin_Backend, IntituteAdmin_Backend, Staff_backend, Student_Backend)
- **ORM:** Prisma, against a single shared PostgreSQL database ("scoms_db")
- **Authentication:** JWT access + refresh tokens, issued by a single unified login handler shared across all four backends
- **Shared library:** an npm workspace package ("shared") providing the Prisma client singleton, auth middleware, error classes, validation schemas, notification senders (email/SMS/WhatsApp), and the three dynamic engines (Report, Certificate, Fee)
- **File/PDF/Excel generation:** pdfkit (PDF), exceljs (Excel), native CSV building

Frontend (×4 — one per portal)

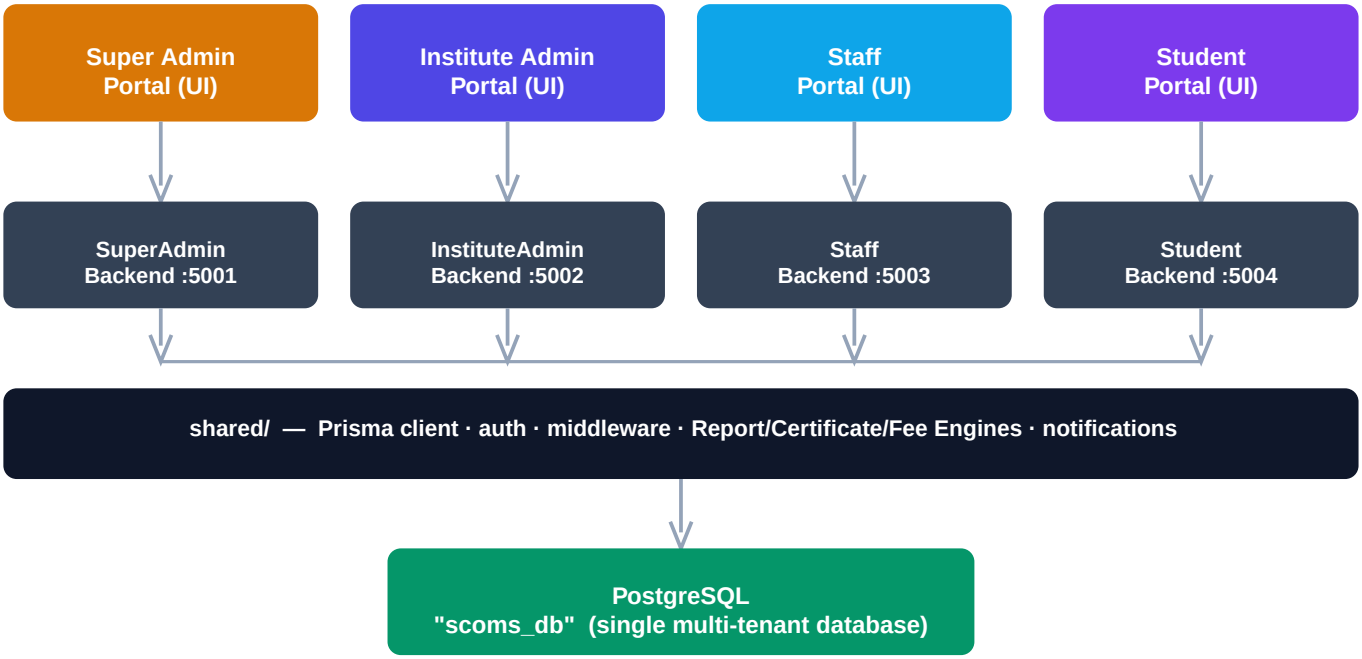
- **Framework:** React 19 with Vite as the build tool
- **Styling:** Tailwind CSS
- **Routing:** React Router, with a per-portal ProtectedRoute guard backed by a real AuthContext
- **State:** React Context (Auth, Institution, Theme, Notifications) — no external state library

Database

One PostgreSQL database, one Prisma schema file, shared by all four backends. Migrations are applied through Prisma's migrate workflow and are fully additive wherever possible — existing tables and data are never destructively altered when a new capability is introduced.

3. System Architecture Diagram

The diagram below shows how the four portals relate to each other, the shared library, and the single database. Each portal is a fully independent web application and API server — there is no cross-portal network traffic; the only thing they share is code (via the "shared" package) and data (via the one Postgres database).



Note the single point of convergence: every backend imports the same shared Prisma client and the same middleware, and every query — no matter which portal issued it — is ultimately scoped by institutionId before it ever reaches the database. This is what makes the multi-tenant isolation guarantee structural rather than something each developer has to remember.

4. Database Design Philosophy

Tenant Isolation

The requireTenant middleware (used by every guarded route in every backend) derives req.institutionId strictly from the authenticated user's JWT — never from a request body, query string, or URL parameter. Every controller function that touches institution-scoped data filters by this value. A student, staff member, or admin from Institution A structurally cannot reach Institution B's rows, because the query itself never asks for them.

Metadata-Driven Engines

Three subsystems — Reports, Certificates, and Fees — follow the same three-layer pattern instead of hard-coding institution-specific behavior into React components or backend logic:

Layer	Purpose	Example Tables
1. Catalog	What exists in SCOMS at all — the system-wide menu of options	Report, CertificateTemplate, FeeHead, FeeCategory
2. Configuration	What one institution has enabled, and how they've customized it	InstitutionReport, InstitutionCertificateTemplate, FeeStructure
3. Engine	The code that merges Layer 1 + Layer 2 and produces a real result	shared/reportEngine, shared/certificateEngine, fee payment/allocation logic

Because of this split, adding a new report type, a new certificate wording, or a new fee head never requires a schema migration or a new React page — it is a new row in a catalog table, immediately available to whichever institutions the Configuration layer enables it for.

Historical Accuracy

Financial and certificate records snapshot their values at the moment of creation (e.g. StudentFeeItem stores feeHeadName and amount independently of the live FeeHead row; Certificate stores renderedBody independently of the live CertificateTemplate). Editing a template or fee head later never silently rewrites a receipt or certificate that was already issued.

5. Module 1 — Super Admin Portal

The Super Admin portal is the platform-operator control plane — it is not scoped to any one institution and is used by SCOMS's own operators, not by school/college staff.

Key Responsibilities

- **Institution Onboarding:** create a new institution tenant, provision its first Institute Admin account, and configure which optional modules (attendance, library, transport, hostel, etc.) are enabled for it
- **Institution Oversight:** view every institution's status, activity, and usage; suspend or reactivate a tenant
- **Report Catalog Management:** create/edit/archive/clone report definitions (the Layer 1 catalog described in Section 4), and assign which institutions can see which reports
- **Certificate Catalog Management:** the equivalent control surface for certificate templates
- **Communication Oversight:** manage institution-wide email/SMS/WhatsApp credit wallets
- **User & Role Management:** platform-level user accounts and role/permission assignment
- **Custom Report Requests:** review requests submitted by Institute Admins for a report that doesn't exist yet, and either build it into the catalog or reject it

Representative Data Model

```
Institution % % < InstitutionReport >% %  Report % % < ReportFilter, ReportColumn
Institution % % < InstitutionCertificateTemplate >% %  CertificateTemplate % % < CertificateField
Institution % % < ReportRequest  (institution !’ super admin review workflow)
```

6. Module 2 — Institute Admin Portal

This is the largest and most feature-dense portal — the day-to-day operating console for one institution's administration. Every page and API call here is scoped to exactly one institutionId, resolved from the logged-in admin's JWT.

6.1 Core Academic & Operational Features

- Institution setup: academic years, departments, courses, semesters, sections, subjects
- Student lifecycle: admission, bulk admission, profile management, promotion between sections, ID cards, alumni records
- Staff lifecycle: registration, qualifications, experience, salary structure, leave, attendance, ID cards
- Attendance oversight: daily register, absentee lists (with SMS resend), institution-wide reports
- Timetable & examination: class/exam timetables, exam scheduling, grade setup, hall tickets, marksheets
- Communication: circulars, email/SMS/WhatsApp broadcast, institution announcements
- Library, transport, hostel, inventory administration

6.2 Dynamic Report Engine

Built to replace what would otherwise be one hard-coded React page per report type. An Institute Admin browses a live catalog of report types (Student List, Attendance Summary/Detailed, Fee Collection, Outstanding Fee, Exam Result, Student Performance, Admission, Institution Summary, and any custom types a Super Admin has added), picks one, and the filter form and result table both render themselves from that report's own database-stored definition — never from hand-written JSX.

Component	Responsibility
Report Catalog	What report types exist and which this institution can see
Report Filters	Each report's own parameter schema — dates, dropdowns cascading from Department !' Course !' Section !' Subject, resolved live and scoped to the institution
Report Generator Engine	Loads the definition, checks permission, validates filters, dispatches to a registered handler function, executes real Prisma queries, logs the run to history
Export	The exact same engine output can become a live preview, a real PDF, a real Excel workbook, or CSV

6.3 Dynamic Certificate Engine

The same three-layer approach applied to certificates. Five default types ship with the system (Bonafide, Transfer & Conduct, Conduct, Course Completion, Fee Paid), each defined as a wording template with {{placeholder}} tokens rather than hard-coded JSX text. An institution can rewrite the wording, header, footer, signatory name/designation, and branding toggles for any certificate type through a "Customize Certificate Format" screen — the change applies to every certificate generated from that point forward, while certificates already issued keep the exact wording they were printed with (a renderedBody snapshot).

6.4 Dynamic Fee Engine

Redesigned so that an institution's fee structure — which fee heads exist, what they cost, how they're grouped into a structure, and how a payment is allocated across them — is entirely database-driven metadata, never a fixed database column per fee type.

```
FeeHead (Tuition, Transport, Exam, ...)  — institution-defined, unlimited
        %
FeeStructure % % < FeeStructureItem >% %  FeeHead   (a named bundle: "2026-27 Grade 10")
        %   DRAFT !’ PUBLISHED !’ (versioned via duplicate) !’ ARCHIVED
        %¼
StudentFeeAssignment % % < StudentFeeItem   (per-student, per-head snapshot at assignment time)
        %¼
FeePayment % % < PaymentAllocation >% %  StudentFeeItem   (one payment can settle many fee heads)
        %¼
FeeReceipt   (printCount tracked — reprinting never alters the original amount or number)
```

A fee structure moves through a real lifecycle — Draft (editable) !’ Published (locked, assignable to students) !’ Archived — and can be duplicated as a new version without breaking the historical assignments that reference the version they were created under. Payments carry an idempotencyKey so a duplicated payment-gateway webhook can never create a second charge.

7. Module 3 — Staff Portal

Scoped to one staff member's own teaching assignments (SubjectTeacherMapping) within their institution — a teacher only ever sees the classes, students, and duties they are actually assigned to.

Key Responsibilities

- **My Classes:** assigned sections/subjects, class rosters, subject mapping, class strength
- **Attendance:** take/edit attendance, monthly grid, subject-wise and institution reports, automatic parent SMS on absence
- **Assignments & Exams:** create assignments, evaluate submissions, question bank (with auto question-paper generation by difficulty), enter and publish exam marks
- **Performance Analytics:** student progress, weak-student/top-performer identification, attendance-vs-marks correlation — all computed from real recorded data, never a placeholder
- **Communication:** SMS broadcast to a class, direct parent contact, message history
- **Events & Lab:** event attendance marking, achievement/certificate entry, lab equipment issue/return ledger
- **Print Templates:** attendance register, assignment evaluation sheets, mark entry sheets, student progress reports, PTM records — all generated from real data, not hard-coded samples

Authentication Note

The Staff portal has its own AuthContext and route guard: any request to a protected page without a valid token redirects to login rather than rendering a shell that then fails every API call — the same pattern used across all four portals.

8. Module 4 — Student & Parent Portal

The student- and parent-facing side of the platform. A student sees only their own academic, financial, and administrative records — every endpoint is scoped by req.user.studentId, derived from the JWT.

Key Responsibilities

- **Academics:** subjects, classroom materials, assignments, exam schedule, results with computed GPA
- **Attendance:** personal attendance summary and history
- **Fees:** fee overview, online payment via a real Razorpay integration with signature verification (not a client-trusted "success" flag), receipts
- **Campus Life:** announcements, events, clubs, digital library, transport, hostel
- **Services:** certificate requests, service requests, grievances, appointments, faculty feedback
- **Career:** resume builder, internships/placements listings, achievements portfolio
- **AI Assistant:** a real backend-logged Q&A assistant (usage tracked for the Super Admin analytics dashboard)
- **Documents:** document vault, digital ID card

Payment Security

Online fee payment never trusts a client-reported "payment succeeded" message. The backend independently verifies the Razorpay order/payment/signature triple via HMAC before marking anything paid, and only then updates the student's fee balance inside a database transaction alongside receipt creation.

9. Security & Multi-Tenancy Model

Authentication

- A single unified login handler (shared/auth/unifiedAuth.js) issues JWT access + refresh tokens for all four portals, encoding { id, email, role, institutionId, studentId, staffId } as applicable
- Every protected route passes through requireAuth (validates the token) and requireTenant (derives req.institutionId strictly from that token)
- Forgot-password / OTP verification / reset flows are shared, real implementations — not placeholders

Authorization

- Role-based access via a shared permission matrix (ROLE_PERMISSIONS)
- Module-level gating: an institution only sees navigation and API routes for modules a Super Admin has enabled for it (requireModule middleware + a module catalog)
- Ownership checks: any endpoint accepting a record id (studentFeeAssignmentId, submissionId, sectionId, etc.) re-verifies that record belongs to the caller's own institution/section/student before acting on it — an id alone is never sufficient authorization

Data Integrity

- Financial mutations happen inside Prisma database transactions — a payment, its allocations, the updated balances, and the receipt are created atomically or not at all
- Idempotency keys prevent a duplicated payment-gateway webhook from creating a second charge
- Soft-delete / archive patterns are used wherever hard-deleting a row could corrupt historical records already referencing it (e.g. deactivating rather than deleting a Fee Head)

10. Glossary

Term	Meaning
Tenant	One institution (school/college) using the platform — data-isolated from every other tenant
Portal	One of the four independent frontend + backend applications (Super Admin, Institute Admin, Staff, Student)
Catalog (Layer 1)	The system-wide list of what report types / certificate types / fee heads exist
Configuration (Layer 2)	One institution's own enable/disable + customization of a catalog item
Engine (Layer 3)	The backend code that merges Layer 1 + Layer 2 and produces a real, live result
Snapshot	A value copied at creation time (e.g. a receipt's amount) so it stays accurate even if the live configuration later changes
Idempotency Key	A client-supplied unique token that lets the backend recognize and ignore a duplicated request (e.g. a retried payment webhook)
JWT	JSON Web Token — the signed credential every authenticated request carries, encoding the user's identity, role, and institution

End of document. This document reflects the SCOMS architecture as of the date on the cover page and is intended as an onboarding and reference guide for engineers, administrators, and stakeholders new to the project.